IntroSort or Introspective sort

Last Updated : 11 Jul, 2025

Introsort(Introspective sort) is a comparison based sort that consists of three sorting phases. They are Quicksort, Heapsort, and Insertion sort. Basic concepts of Introsort and the C++ code are available here
The following section shows how the Introsort algorithm is formulated, after reviewing the pros and cons of the respective algorithms.

1. Quicksort
   The Quicksort is an efficient sorting algorithm but has the worst-case performance of O(N ^ 2) comparisons with O(N) auxiliary space. This worst-case complexity depends on two phases of the Quicksort algorithm.
   1. Choosing the pivot element
   2. Recursion depth during the course of the algorithm
2. Heapsort
   Heapsort has O(N log N) worst-case time complexity, which is much better than the worst case of Quicksort. So, is it evident that Heapsort is the best? No, the secret of Quicksort is that it does not swap already elements that are already in order, which is unnecessary, whereas with the Heapsort, even if all of the data is already sorted, the algorithm swaps all of the elements to order the array. Also, by choosing the optimal pivot, the worst-case of O(N ^ 2) can be avoided in quicksort. But, the swapping will pay more time in the case of Heapsort that is unavoidable. Hence, Quicksort outperforms Heapsort.
   The best things about Heapsort is that, if the recursion depth becomes too large like (log N), the worst case complexity would be still O(N log N).
3. Mergesort
   The Mergesort has the worst case complexity only as O(N log N). Mergesort can work well on any type of data sets irrespective of its size whereas the Quicksort cannot work well with large data sets. But, Mergesort is not in-place whereas Quicksort is in-place, and that plays a vital role in here. Mergesort goes well with LinkedLists whereas Quicksort goes well with arrays. The locality of reference is better with Quicksort, whereas with Mergesort it is bad. So, for conventional purposes, having memory constraints in hand, Quicksort outperforms Mergesort.
4. Insertion sort
   The main advantage of the insertion sort is its simplicity. It also exhibits a good performance when dealing with a small list. The insertion sort is an in-place sorting algorithm so the space requirement is minimal. The disadvantage of the insertion sort is that it does not perform as well as the other sorting algorithms when the size of the data gets larger.

Here is how Introsort is formulated:
Choosing the right sorting algorithm depends on the occasion where the sorting algorithm is being used. There are a good number of sorting algorithms in hand already that has pros and cons of its own. So, to get a better sorting algorithm, the solution is to tweak the existing algorithms and produce a new sorting algorithm that works better. There are a lot of hybrid algorithms, that outperforms the general sorting algorithms. One such is the Introsort. The best versions of Quicksort are competitive with both heap sort and merge sort on the vast majority of inputs. Rarely Quicksort has the worst case of O(N ^ 2) running time and O(N) stack usage. Both Heapsort and Mergesort have O(N log N) worst-case running time, together with a stack usage of O(1) for Heapsort and O(log N) for Mergesort respectively. Also, Insertion sort performs better than any of the above algorithms if the data set is small.
Combining all the pros of the sorting algorithms, Introsort behaves based on the data set.

1. If the number of elements in the input gets fewer, the Introsort performs Insertion sort for the input.
2. Having the least number of comparisons(Quicksort) in mind, for splitting the array by finding the pivot element, Quicksort is used. Quoted earlier, the worst case of Quicksort is based on the two phases and here is how we can fix them.
   1. Choosing the pivot element: We can use either of median-of-3 concept or randomized pivot concept or middle as the pivot concept for finding the pivot element
   2. Recursion depth during the course of the algorithm: When the recursion depth gets higher, Introsort uses Heapsort as it has the definite upper bound of O(N log N).

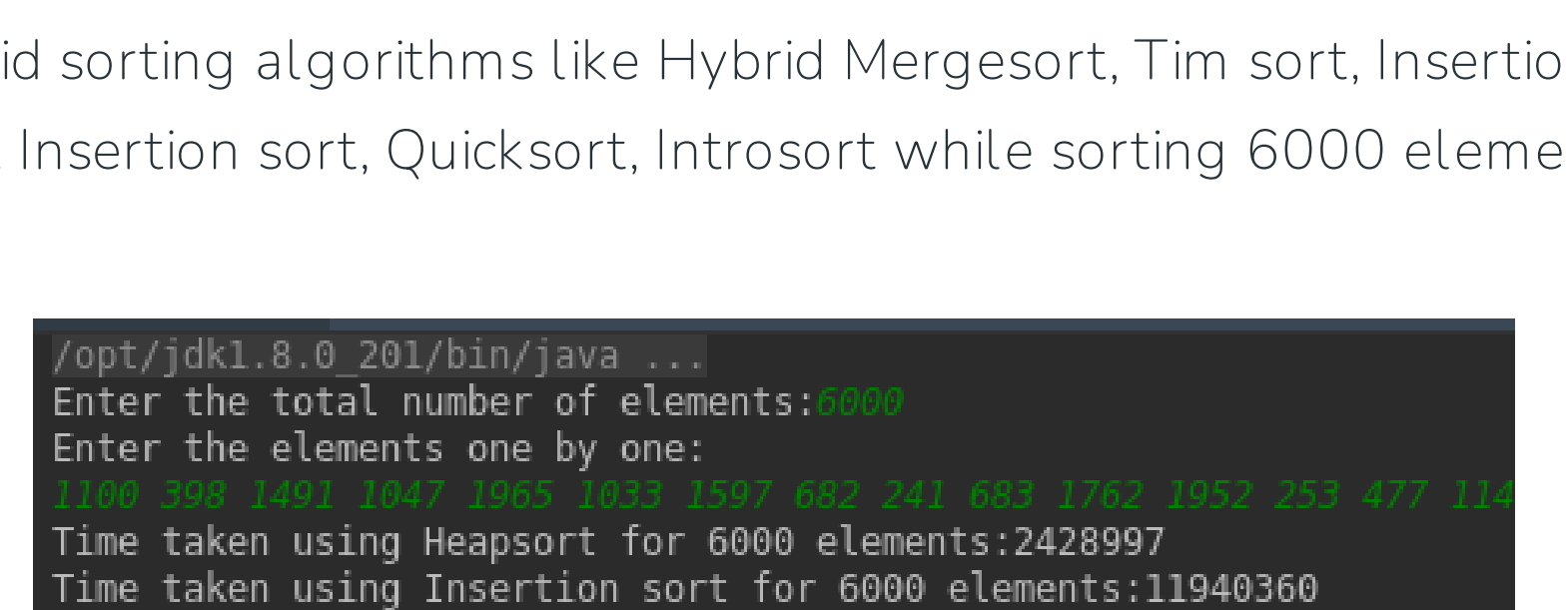
How does depthLimit work?
depthLimit represents maximum depth for recursion. It is typically chosen as log of length of input array (please refer below implementation). The idea is to ensure that the worst case time complexity remains O(N log N). Note that the worst-case time complexity of HeapSort is O(N log N).

Why is Mergesort not used?
As the arrays are being dealt with the in-place concept where Quicksort outperforms Mergesort, we are not using Mergesort.

Can Introsort be applied everywhere?

1. If the data won't fit in an array, Introsort cannot be used.
2. Furthermore, like Quicksort and Heapsort, Introsort is not stable. When a stable sort is needed, Introsort cannot be applied.

Is Introsort the only hybrid sorting algorithm?
No. There are other hybrid sorting algorithms like Hybrid Mergesort, Tim sort, Insertion-Merge hybrid.
Comparison of Heapsort, Insertion sort, Quicksort, Introsort while sorting 6000 elements(in milliseconds).



```
/opt/jdk1.8.0_201/bin/java ...
Enter the total number of elements:6000
Enter the elements one by one:
1100 398 1491 1047 1965 1033 1597 682 241 683 1762 1952 253 477 114
Time taken using Heapsort for 6000 elements:2428997
Time taken using Insertion sort for 6000 elements:11940360
Time taken using Quicksort for 6000 elements:2220502
Time taken using Introsort for 6000 elements:1079874

Process finished with exit code 0
```

Pseudocode:

```
sort(A : array):
    depthLimit = 2xfloor(log(length(A)))
    introsort(A, depthLimit)

introsort(A, depthLimit):
    n = length(A)
    if n<=16:
        insertionSort(A)
    if depthLimit == 0:
        heapsort(A)
    else:

        // using quick sort, the
        // partition point is found
        p = partition(A)
        introsort(A[0:p-1], depthLimit - 1)
        introsort(A[p+1:n], depthLimit - 1)
```

Time Complexity:
Worst-case performance: O(nlogn) (better than Quicksort)
Average-case performance: O(nlogn)
In the Quicksort phase, the pivot can either be chosen using the median-of-3 concept or last element of the array. For data that has a huge number of elements, median-of-3 concept slows down the running time of the Quicksort.
In the example described below, the quicksort algorithm calculates the pivot element based on the median-of-3 concept.

Example:

C++ Java C# JavaScript Python3

```cpp
// C++ implementation of Introsort algorithm

#include <bits/stdc++.h>
using namespace std;

// A utility function to swap the values pointed by
// the two pointers
void swapValue(int* a, int* b)
{
    int* temp = a;
    a = b;
    b = temp;
    return;
}

/* Function to sort an array using insertion sort*/
void InsertionSort(int arr[], int* begin, int* end)
{
    // Get the left and the right index of the subarray
    // to be sorted
    int left = begin - arr;
    int right = end - arr;

    for (int i = left + 1; i <= right; i++) {
        int key = arr[i];
        int j = i - 1;

        /* Move elements of arr[0..i-1], that are
                greater than key, to one position ahead
                of their current position */
        while (j >= left && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }

    return;
}

// A function to partition the array and return
// the partition point
int* Partition(int arr[], int low, int high)
{
    int pivot = arr[high]; // pivot
    int i = (low - 1); // Index of smaller element

    for (int j = low; j <= high - 1; j++) {
        // If current element is smaller than or
        // equal to pivot
        if (arr[j] <= pivot) {
            // increment index of smaller element
            i++;

            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return (arr + i + 1);
}

// A function that find the middle of the
// values pointed by the pointers a, b, c
// and return that pointer
int* MedianOfThree(int* a, int* b, int* c)
{
    if (*a < *b && *b < *c)
        return (b);

    if (*a < *c && *c <= *b)
        return (c);

    if (*b <= *a && *a < *c)
        return (a);

    if (*b < *c && *c <= *a)
        return (c);

    if (*c <= *a && *a < *b)
        return (a);

    if (*c <= *b && *b <= *a)
        return (b);
}

// A Utility function to perform intro sort
void IntrosortUtil(int arr[], int* begin, int* end,
                   int depthLimit)
{
    // Count the number of elements
    int size = end - begin;

    // If partition size is low then do insertion sort
    if (size < 16) {
        InsertionSort(arr, begin, end);
        return;
    }

    // If the depth is zero use heapsort
    if (depthLimit == 0) {
        make_heap(begin, end + 1);
        sort_heap(begin, end + 1);
        return;
    }

    // Else use a median-of-three concept to
    // find a good pivot
    int* pivot
        = MedianOfThree(begin, begin + size / 2, end);

    // Swap the values pointed by the two pointers
    swapValue(pivot, end);

    // Perform Quick Sort
    int* partitionPoint
        = Partition(arr, begin - arr, end - arr);
    IntrosortUtil(arr, begin, partitionPoint - 1,
                  depthLimit - 1);
    IntrosortUtil(arr, partitionPoint + 1, end,
                  depthLimit - 1);

    return;
}

/* Implementation of introsort*/
void Introsort(int arr[], int* begin, int* end)
{
    int depthLimit = 2 * log(end - begin);

    // Perform a recursive Introsort
    IntrosortUtil(arr, begin, end, depthLimit);

    return;
}

// A utility function ot print an array of size n
void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " \n"[i + 1 == n];
}

// Driver program to test Introsort
int main()
{
    int arr[] = { 2,  10, 24, 2,  10, 11, 27, 4,  2,  4,
                  28, 16, 9,  8,  28, 10, 13, 24, 22, 28,
                  0,  13, 27, 13, 3,  23, 18, 22, 8,  8 };
    int n = sizeof(arr) / sizeof(arr[0]);

    // Pass the array, the pointer to the first element and
    // the pointer to the last element
    Introsort(arr, arr, arr + n - 1);
    printArray(arr, n);

    return (0);
}
```

Output

```
0 2 2 2 3 4 4 8 8 8 9 10 10 10 11 13 13 13 16 18 22 22 23 24 24 27 27 28 28 28
```

Comment   L lokesh ...   16

Explore

DSA Fundamentals
Data Structures
Algorithms
Advanced
Interview Preparation
Courses

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 6th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

GET IT ON Google Play   Download on the App Store

Company
About Us
Legal
Privacy Policy
Contact Us
Advertise with us
GFG Corporate Solution
Campus Training Program

Explore
POTD
Job-A-Thon
Blogs
Nation Skill Up

Tutorials
Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

Courses
ML and Data Science
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos
DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner
Interview Corner
Aptitude
Puzzles
GfG 160
System Design

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved